

Hacking with Linux networking command line tools

Xiaolin Wang

2025年12月16日

目录

1 Caution	1
1.1 Deadline: [2025-12-21 Sun]	3
2 tmux, nc, ip, tcpdump, ss, nmap, curl	3
2.1 Your tasks	4
3 Socket programming	4
3.1 TCP	4
3.1.1 A simple TCP server written in Python3	4
3.1.2 A simple TCP client written in Python3	5
3.1.3 A simple TCP demo script	5
3.2 UDP	6
3.2.1 A simple UDP server written in Python3	6
3.2.2 A simple UDP client written in Python3	6
3.2.3 A simple UDP demo script	7

1 Caution

You must submit your report as a tar ball in which the following files should be included in:

- Your report in either Emacs Org or Markdown format, and a PDF file generated from your org or md file.

Tips:

- Report template: org file, markdown file, yaml file, Makefile, and the generated pdf file.
- In Emacs org-mode, press `C-c C-e l p` to export a PDF file.

- For Markdown to PDF, you can simply do make pdf if you are using Linux in lab 215 and have these yaml file and Makefile along with your md file.
 - Your lab journal (example).
 - Your source codes (examples).
 - A screencast file in ttyrec format recording your operations (man ttyrec).
- Here' s how:

-
1. make a directory, e.g. 20231152xxx, in this dir try very hard to make all the files available.

```
1 mkdir 20231152xxx      # create a new directory
2 cd 20231152xxx
3 vim tcpServer.c        # Implement the TCP server in C
4 vim tcpClient.c        # Implement the TCP client in C
5 # compile and run above C programs successfully, and then
6
7 # do the HTTP demo
8 vim tmux-http.sh
9
10 # record a screencase using ttyrec (in a plain terminal without tmux)
11 ttyrec -f http-demo.ttyrec -- tmux new
12 # play screencast
13 ttyplay http-demo.ttyrec
14
15 # make sure you successfully finished all the above, then
16 # write your report
17 vim 20231152xxx.yaml # write a yaml file, and
18 vim 20231152xxx.md   # your report in markdown format
19
20 # generate the PDFs from the above yaml and md files
21 # (make sure you have the Makefile ready)
22 make report_pdf
23 make journal_pdf
24
25 # in case you prefer emacs to vi
26 emacs 20231152xxx.org # report in emacs-org format
27 # in emacs type C-c C-e l p to generate a PDF
```

2. make a tar ball.

```
1 cd ..
2 tar zcf 20231152xxx.tgz 20231152xxx
3 ls -l # make sure your tar ball is smaller than 1MB in size
```

3. submit the tgz file to our moodle site.

Here is a short tutorial about writing lab report: tutorial.ttyrec. To view it:

```
1 ttyplay tutorial.ttyrec
```

Feel free to make your own ttyrec files while doing this lab work. For example:

```
ttyrec -f 20231152xxx-http.ttyrec -- tmux new
ttyrec -f 20231152xxx-email.ttyrec -- tmux new
ttyrec -f 20231152xxx-ftp.ttyrec -- tmux new
```

Bonus point Manage your project with git. man gittutorial to learn the very basics of it.

1.1 Deadline: [2025-12-21 Sun]

- Submit your report as a tgz file here. In your tgz file, there must be:
 - your report in org or markdown format.
 - your report in PDF format.
 - your bash script for demonstrating a HTTP session.
 - one or more ttyrec files for demonstrating whatever you did.
- All the paperwork templates and examples can be found here.
- Late reports will be penalized 20% per day.
- MS-word file will **NOT** be accepted. Cheating will result in automatic failure of this work.

2 tmux, nc, ip, tcpdump, ss, nmap, curl

Here are the bash scripts I used in the class for demonstrating how some protocols work.

- TCP three-way handshake
- UDP
- SMTP (need a SMTP server)
- FTP (need a FTP server)

2.1 Your tasks

1. Run the above scripts to get familiar with these tools, and get a thorough understanding about these protocols;
2. Packet analysis. Upon running the following command:

```
1 sudo tcpdump -ilo -nnvvvXKS -s0 port 3333
```

the following packet is captured:

```
08:34:10.790666 IP (tos 0x0, ttl 64, id 12824, offset 0, flags [DF],
proto TCP (6), length 64)
127.0.0.1.46668 > 127.0.0.1.3333: Flags [P.], seq 2400005725:2400005737,
ack 373279396, win 512, options [nop,nop,TS val 3259949783 ecr 3259896343],
length 12
 0x0000: 4500 0040 3218 4000 4006 0a9e 7f00 0001  E..@2.@.....
 0x0010: 7f00 0001 b64c 0d05 8f0d 2e5d 163f caa4  .....L.....].?..
 0x0020: 8018 0200 fe34 0000 0101 080a c24e e2d7  ....4.....N..
 0x0030: c24e 1217 6865 6c6c 6f20 776f 726c 640a  .N..hello.world.
```

- (a) Tell me the meaning of each option used in the previous command.
 - (b) Please analyze this captured packet and explain it to me as detailed as you can.
3. Write a similar script showing how HTTP works (you need curl).
 4. Record your HTTP demo session with `ttyrec`.

3 Socket programming

The followings are the Python programs I used in the class for demonstrating socket programming. Your tasks

1. Try these programs with a remote server IP instead of 127.0.0.1.
2. Rewrite them in C.

3.1 TCP

3.1.1 A simple TCP server written in Python3

```
1 #!/usr/bin/python3
2
```

```

3  ### A simple TCP server ###
4
5  from socket import *
6  serverPort = 12000
7  serverSocket = socket(AF_INET,SOCK_STREAM)
8  serverSocket.bind(('',serverPort))
9  serverSocket.listen(0)
10 print(serverSocket.getsockname())
11 print('The server is ready to receive')
12 while 1:
13     connectionSocket, addr = serverSocket.accept()
14     print(connectionSocket.getsockname())
15     sentence = connectionSocket.recv(1024)
16     capitalizedSentence = sentence.upper()
17     connectionSocket.send(capitalizedSentence)
18     connectionSocket.close()

```

3.1.2 A simple TCP client written in Python3

```

1  #!/usr/bin/python3
2
3  ### A simple TCP client ###
4
5  from time import *
6  from socket import *
7  serverName = '127.0.0.1'
8  serverPort = 12000
9  clientSocket = socket(AF_INET, SOCK_STREAM)
10 clientSocket.connect((serverName,serverPort))
11 print(clientSocket.getsockname())
12 sentence = input('Input lowercase sentence:')
13 clientSocket.send(bytes(sentence,'utf-8'))
14 modifiedSentence = clientSocket.recv(1024)
15 print('From Server:', str(modifiedSentence,'utf-8'))
16 clientSocket.close()

```

3.1.3 A simple TCP demo script

```

1  #!/bin/bash
2
3  ### A simple TCP demo script ###
4
5  set -euC
6
7  tmux rename-window "TCP demo"
8

```

```

9 # Window setup
10 # +-----+-----+
11 # | server | client |
12 # +-----+-----+
13 # |   watch   |
14 # +-----+
15 # |   tcpdump   |
16 # +-----+
17 #
18 tmux split-window -h
19 tmux split-window -f99
20 tmux split-window -l12
21
22 tmux send-keys -t{top-left} "./tcpServer.py"
23
24 tmux send-keys -t{top-right} "./tcpClient.py"
25
26 tmux send-keys -t{up-of} "watch -tn.1 'ss -ant \"( sport == 12000 or dport ==
↵ 12000 )\"'" C-m
27
28 tmux send-keys "sudo tcpdump -ilo -vvvnxXSK -s0 port 12000" C-m

```

3.2 UDP

3.2.1 A simple UDP server written in Python3

```

1 #!/usr/bin/python3
2
3 ### A simple UDP server ###
4
5 from socket import *
6 serverPort = 12000
7 serverSocket = socket(AF_INET, SOCK_DGRAM)
8 serverSocket.bind(('', serverPort))
9 print("The server is ready to receive")
10 while 1:
11     message, clientAddress = serverSocket.recvfrom(2048)
12     modifiedMessage = message.upper()
13     serverSocket.sendto(modifiedMessage, clientAddress)

```

3.2.2 A simple UDP client written in Python3

```

1 #!/usr/bin/python3
2
3 ### A simple UDP client ###
4

```

```

5 from socket import *
6 serverName = '127.0.0.1'
7 serverPort = 12000
8 clientSocket = socket(AF_INET, SOCK_DGRAM)
9 message = input('Input lowercase sentence:')
10 clientSocket.sendto(bytes(message,'utf-8'),(serverName, serverPort))
11 modifiedMessage, serverAddress = clientSocket.recvfrom(2048)
12 print(str(modifiedMessage,'utf-8'))
13 clientSocket.close()

```

3.2.3 A simple UDP demo script

```

1 #!/bin/bash
2
3 ### A simple UDP demo script ###
4
5 set -euC
6
7 tmux rename-window "UDP demo"
8
9 # Window setup
10 # +-----+-----+
11 # | server | client |
12 # +-----+-----+
13 # | tcpdump |
14 # +-----+
15 #
16 tmux split-window -h
17 tmux split-window -f99
18
19 tmux send-keys -t{top-left} "./udpServer.py"
20 tmux send-keys -t{top-right} "./udpClient.py"
21
22 tmux send-keys "sudo tcpdump -ilo -vvvnxXK port 12000" C-m

```
